

Ce problème est extrait du sujet **Informatique I** de **Mines-Ponts MPI 2023** (également prévu pour être traité en 3h). La principale différence est l'absence des deux dernières questions, qui font référence au programme de 2ème année. Quelques indications supplémentaires et clarifications ont été ajoutées, et certaines questions ont été décomposées.

Introduction

Présentation du problème

Ce problème comporte trois parties ; les parties 1 et 2 sont indépendantes. La partie 3 utilise des résultats des parties précédentes qui pourront au besoin être admis, et des fonctions des parties précédentes que l'on supposera implémentées.

L'objectif du problème est de construire des *listes à accès direct* : une liste à accès direct est un type de donnée abstrait qui permet, d'une part, d'empiler et de dépiler efficacement un élément en tête de liste (comme une liste chaînée ou une pile) et, d'autre part, d'accéder efficacement au k^{e} élément de la liste, pour n'importe quel indice k valide (comme un tableau).

Dans la première section, nous étudions un système de numération : la représentation binaire gauche des entiers naturels. Dans la deuxième section, nous étudions les arbres binaires parfaits. Dans la troisième section, nous implémentons le type liste à accès direct par la structure de données concrète *liste gauche*, que nous construisons en exploitant les résultats obtenus dans les sections précédentes.

Dans tout l'énoncé, un même identificateur écrit dans deux polices de caractère différentes désignera la même entité, mais du point de vue mathématiques avec la police en italique (par exemple n) et du point de vue informatique avec celle en romain avec espacement fixe (par exemple `n`).

Travail attendu

Lorsqu'une fonction est demandée, celle-ci doit selon la question être écrite en C exclusivement, ou bien, si le sujet le précise, en pseudo-code utilisant une syntaxe plus souple mais conforme aux possibilités offertes par le langage C.

Une grande importance sera apportée à la clarté et la lisibilité de vos fonctions. N'hésitez pas à les accompagner d'explications si nécessaire. Il n'est pas nécessaire d'écrire la spécification d'une fonction quand elle est donnée dans l'énoncé, mais c'est recommandé pour toute fonction auxiliaire que vous utiliserez.

On pourra toujours utiliser une fonction demandée à une question précédente, même si cette question n'a pas été traitée.

Attention à bien lire les définitions données dans l'énoncé !

Conventions et rappels

Les bibliothèques `stdbool.h`, `assert.h`, `stdlib.h`, `stdio.h` et `math.h` peuvent comme toujours être supposées incluses sans rappel sur la copie.

On utilisera tout du long uniquement le type `int` pour représenter les entiers, et on supposera qu'il n'est jamais sujet à des débordements.

On rappelle que l'expression `1 << n` représente le décalage de la valeur 1 sur n bits vers la gauche : elle a pour valeur l'entier 2^n . Contrairement à `lsl` en OCaml, **cet opérateur n'est pas prioritaire sur les opérateurs arithmétiques usuels**. Ainsi `1 << 5 + 10` est évalué à $2^{15} = 32768$ et non pas $2^5 + 10 = 42$.

1 Représentation binaire gauche des entiers naturels

1.1 Mise en place

Soient m un entier naturel et N un entier naturel non nul. Il est classique d'appeler *représentation binaire standard de l'entier m sur N chiffres*, ou plus simplement *représentation standard*, toute suite finie $b = (b_n)_{0 \leq n < N}$ de longueur N telle que, pour tout indice n compris entre 0 et $N - 1$, le chiffre b_n appartient à $\{0, 1\}$ et l'égalité suivante est vérifiée

$$m = \sum_{n=0}^{N-1} b_n 2^n.$$

Définition : Nous appelons *représentation binaire gauche de l'entier m sur N chiffres* toute suite finie $g = (g_n)_{0 \leq n < N}$ de longueur N telle que,

- pour tout indice n compris entre 0 et $N - 1$, le chiffre g_n appartient à $\{0, 1, 2\}$,
- l'égalité suivante est satisfaite

$$m = \sum_{n=0}^{N-1} g_n (2^{n+1} - 1),$$

- le chiffre « 2 » n'apparaît qu'au plus une fois parmi les chiffres $(g_n)_{0 \leq n < N}$,
- s'il existe une position p telle que le chiffre g_p est 2, alors, pour tout indice n compris entre 0 et $p - 1$, le chiffre g_n est nul.

De manière plus courte, nous parlons simplement de *représentation gauche*.

La figure 1 ci-dessous donne une représentation standard et une représentation gauche sur quatre chiffres des seize premiers entiers. Conformément à l'usage habituel, nous écrivons toute représentation, qu'elle soit standard ou gauche, sous la forme d'un mot $b_{N-1} \cdots b_0$ ou $g_{N-1} \cdots g_0$ dans lequel les chiffres de poids faibles sont écrits à droite.

Entier	Repr. standard	Repr. gauche	Entier	Repr. standard	Repr. gauche
0	0000	0000	8	1000	0101
1	0001	0001	9	1001	0102
2	0010	0002	10	1010	0110
3	0011	0010	11	1011	0111
4	0100	0011	12	1100	0112
5	0101	0012	13	1101	0120
6	0110	0020	14	1110	0200
7	0111	0100	15	1111	1000

FIGURE 1 – Représentations standard et gauche des seize premiers entiers

1. Soit c un entier. Donner la représentation standard de l'entier dont une représentation gauche est $10 \cdots 0$ (avec c chiffres nuls) en justifiant sommairement. Faire de même avec l'entier dont une représentation gauche est $20 \cdots 0$ (avec c chiffres nuls).
2. Déterminer, en justifiant, le plus grand entier naturel M_N qui admet une représentation gauche sur N chiffres. Préciser la représentation gauche de cet entier.

Définition : Soit n_0 un indice compris entre 0 et $N - 1$. On dit que l'indice n_0 est la *position du chiffre de plus fort poids* d'une représentation $g_{N-1} \cdots g_0$ si l'indice n_0 est le plus petit entier tel que, pour tout indice $n > n_0$, le chiffre g_n est nul. On appelle le chiffre g_{n_0} le *chiffre de plus fort poids*.

3. Soient N un entier naturel non nul, $g = g_{N-1} \cdots g_0$ et $h = h_{N-1} \cdots h_0$ deux représentations gauches d'un même entier m . Démontrer que les chiffres de plus fort poids de g et de h sont de même valeur et à la même position.

4. Démontrer que tout entier appartenant à l'intervalle $\llbracket 0, M_N \rrbracket$, où l'entier M_N a été introduit à la question 2, ne possède au plus qu'une seule représentation gauche sur N chiffres.

Pour la suite de cette partie, l'entier N est déclaré comme constante globale avec **define**. Nous utilisons la structure déclarée comme suit pour écrire la représentation gauche sur N chiffres d'un entier $g_{N-1} \cdots g_0$:

```
1 #define N 8
2
3 struct RepGauche {
4     int position;
5     bool chiffres[N];
6 };
7
8 typedef struct RepGauche rg;
```

Le champ **position** repère la position éventuelle du chiffre 2 ; il vaut -1 au cas où le chiffre 2 n'apparaît pas. Pour tout indice n compris entre 0 et $N - 1$, la n^{e} case du champ **chiffres** vaut **true** si le chiffre g_n est non-nul et vaut **false** sinon.

Par exemple, les entiers 15 et 21 ont pour représentation gauche les variables **entier_15** et **entier_21** suivantes :

```
1 rg entier_15 =
2     { position = -1,
3       chiffres = { false, false, false, true, false, false, false, false }
4     };
5
6 rg entier_21 =
7     { position = 1,
8       chiffres = { false, true, false, true, false, false, false, false }
9     };
```

Pour tout **c** de type **rg**, **c** correspond à la représentation gauche d'un entier tel que définie plus haut si et seulement si :

- **c.position** $\in \llbracket -1; N - 1 \rrbracket$
 - Si **c.position** ≥ 0 , **c.chiffres[c.position]** est **true**.
 - Si **c.position** ≥ 0 , pour tout $i \in \llbracket 0; \text{c.position} - 1 \rrbracket$, **c.chiffres[i]** est **false**.
5. Écrire en C une fonction **int rg_to_int(rg g)**, qui renvoie l'entier dont g est la représentation gauche. On supposera que **g** est bien une représentation gauche valide.

1.2 Incrémentation et décrémentation

Nous proposons l'algorithme suivant :

ALGORITHME MYSTÈRE :

Entrée : Représentation gauche $g = g_{N-1} \cdots g_0$ d'un certain entier m .

Effet :

- Si aucun des chiffres $(g_n)_{0 \leq n < N}$ ne vaut 2, changer le chiffre g_0 en $g_0 + 1$.
- Sinon, en notant p la position du chiffre 2, changer le chiffre g_p en 0 et le chiffre g_{p+1} en $g_{p+1} + 1$.

Nous notons m' l'entier dont la représentation gauche est g après exécution de l'algorithme.

FIGURE 2 – Un algorithme

6. (a) Montrer que si **g** est une représentation gauche valide avant l'appel, alors c'est toujours le cas après l'exécution de l'algorithme mystère (cf. figure 2).
- (b) Avec les notations m et m' de la figure 2, caractériser, en fonction de l'entier m , la valeur de l'entier m' .

7. Écrire en C une fonction `bool rg_incr(rg* s)` dont la spécification suit :
- *Précondition* : La variable `s` est un pointeur vers la représentation gauche d'un entier m .
 - *Effet de bord* : La valeur pointée par `s` est modifiée afin de représenter l'entier $m + 1$.
 - *Valeur de retour* : Booléen `true` si l'incrément de m a eu lieu correctement et `false` si un débordement se produit car $m + 1$ n'est pas représentable sur avec N chiffres.
8. Calculer la complexité en temps dans le pire des cas de la fonction `rg_incr` en fonction de N . Comparer avec la complexité de l'incrément sur la représentation binaire usuelle.
9. Écrire une fonction `bool rg_decr(rg* s)` dont la spécification suit. On pourra se contenter de pseudo-code.
- *Précondition* : Le pointeur `s` désigne la représentation gauche d'un entier m .
 - *Effet de bord* : La valeur pointée par `s` est modifiée afin de représenter l'entier $m - 1$.
 - *Valeur de retour* : Booléen `true` si la décrémentation de m a eu lieu correctement et `false` si un débordement se produit car m est nul.

Il est recommandé d'expliquer son intention avant le code de la fonction.

2 Arbres binaires parfaits

2.1 Opérations sur les arbres binaires

Nous représentons les arbres binaires à valeurs entières au moyen du type `arb*`, qui est un pointeur vers une structure contenant la valeur entière dans le champ `valeur` et les deux enfants dans les champs `gauche` et `droit`. L'arbre vide se représente par le pointeur `NULL`.

```

1 typedef struct Noeud arb;
2 struct Noeud {
3     int valeur;
4     arb* gauche;
5     arb* droit;
6 };

```

L'arbre vide est, par convention, de hauteur -1 .

On admettra que tous les pointeurs de type `arb*` qu'on manipule correspondent bien à un arbre binaire, c'est à dire qu'un nœud n'est jamais réutilisé dans un même arbre.

10. Écrire en C une fonction `int hauteur(arb* a)` qui calcule la hauteur de l'arbre a .
11. Écrire en C une fonction `arb* noeud(int v, arb* a_g, arb* a_d)` qui construit un arbre dont la racine a pour étiquette v , dont l'enfant gauche est a_g et l'enfant droit est a_d .

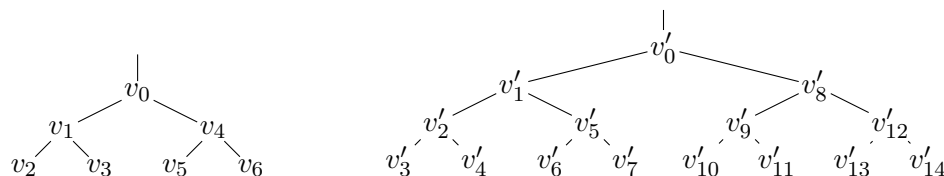


FIGURE 3 – Arbres binaires parfaits de hauteur 2 et de hauteur 3.

2.2 Arbres parfaits

Définition : Un *arbre binaire parfait*, ou simplement *arbre parfait*, est un arbre binaire dont tous les nœuds internes ont exactement deux fils, dont toutes les feuilles sont à la même profondeur et dont les nœuds sont étiquetés par des valeurs entières.

Des exemples d'arbres binaires parfaits sont donnés figure 3.

12. Démontrer que tout arbre binaire parfait de hauteur n possède $2^{n+1} - 1$ nœuds.
13. Écrire en C une fonction `bool est_parfait(arb* a, int n)` dont la spécification suit :
 - *Précondition* : Le pointeur a désigne un arbre binaire.
 - *Valeur de retour* : Booléen `true` si l'arbre pointé est parfait de hauteur n et `false` sinon.
14. Calculer la complexité en temps dans le pire des cas de l'exécution de `est_parfait(a, n)` en fonction de l'entier n .

Si cette complexité n'est pas bien définie dans votre fonction, justifiez-le, et donnez une complexité en fonction d'un paramètre qui vous semble pertinent.

2.3 Opérations sur les arbres parfaits

Définition : Soient S une structure de données qui permet de stocker une collection d'entiers et t le cardinal de S . Nous disons que la structure de données S est à *accès direct* s'il existe une manière systématique de numéroter chaque élément de S entre 0 et $t-1$ et s'il existe deux fonctions, `acces` et `modif`, qui permettent respectivement de consulter et de modifier n'importe quel élément de S en temps logarithmique en t à partir seulement de son numéro.

Par exemple, un tableau est à accès direct, alors qu'une liste chaînée ne l'est pas.

Nous souhaitons vérifier qu'un arbre parfait est une structure de données à accès direct. Nous numérotions les éléments d'un arbre parfait en utilisant l'ordre préfixe de gauche à droite de l'arbre (comme illustré dans la figure 3).

15. Écrire en C une fonction `arb* arb_trouve(arb* a, int n, int k)` ainsi spécifiée :
 - *Précondition* : Le pointeur a désigne la racine d'un arbre binaire (supposé non vide) parfait de hauteur $n \geq 0$. L'entier k satisfait les inégalités $0 \leq k < 2^{n+1} - 1$.
 - *Valeur de retour* : Pointeur vers le k^e nœud de l'arbre dans l'ordre préfixe.

Il est rappelé que la racine d'un arbre est à profondeur 0. Nous donnons la formule de sommation, valable pour tout réel $a \neq 1$ et tout entier t :

$$\sum_{k=0}^t k \cdot a^k = \frac{((a-1)t-1)a^{t+1} + a}{(a-1)^2}$$

On rappelle également que l'espérance d'une variable aléatoire X dans $\llbracket 0; t \rrbracket$ est $E(X) = \sum_{k=0}^t P(X=k)k$.

16. Nous appelons P la profondeur d'un nœud choisi aléatoirement et uniformément parmi les $2^{n+1} - 1$ nœuds d'un arbre binaire parfait de hauteur n . Calculer l'espérance $\mathbb{E}(P)$ de la variable aléatoire P .
17. Déterminer la complexité (en temps) moyenne (c'est à dire, l'espérance de la complexité) de l'instruction `arb_trouve(a, n, k)` lorsque la hauteur n et l'arbre a sont fixés et le numéro k est choisi aléatoirement et uniformément dans l'intervalle $\llbracket 0, 2^{n+1} - 2 \rrbracket$.
18. Écrire en C :
 - (a) une fonction `int arb_acces(arb* a, int n, int k)` qui renvoie la k^e valeur de l'arbre a de hauteur n ;
 - (b) une fonction `void arb_modif(arb* a, int n, int k, int v)` qui remplace la k^e valeur de l'arbre a de hauteur n par la valeur v .
19. Justifier que la complexité dans le pire cas de ces fonctions est logarithmique en la taille de la structure, et que les arbres binaires parfaits forment bien une structure à accès direct.

3 Listes gauches

Les arbres binaires parfaits seuls ne permettent de représenter que des collections d'entiers dont le cardinal est de la forme $2^{n+1} - 1$ où n est entier naturel. Afin de représenter des collections de cardinal quelconque, nous introduisons des suites d'arbres parfaits, aux hauteurs strictement croissantes et savamment choisies.

Définition : Nous appelons *liste binaire gauche de cardinal m sur N arbres* la structure de données constituée de $N + 1$ arbres binaires parfaits $((a_n)_{0 \leq n < N}, e)$ comme suit. Nous décomposons l'entier m selon sa représentation binaire gauche sur N chiffres : $g_{N-1} \cdots g_0$. Pour tout indice n compris entre 0 et $N - 1$, si le chiffre g_n n'est pas nul, l'arbre binaire parfait a_n est un arbre de hauteur n , sinon il s'agit de l'arbre vide. Si le chiffre 2 apparaît parmi les chiffres de la représentation gauche de m et si l'indice p est sa position, alors l'arbre e est un arbre binaire parfait de hauteur p , sinon l'arbre e est l'arbre vide. De manière plus courte, nous parlons simplement de *liste gauche*.

Une liste binaire gauche¹ de cardinal m sur N arbres permet de stocker une collection de m éléments entiers. Il apparaît qu'avec N arbres, une liste binaire gauche peut stocker jusqu'à M_N entiers (où l'entier M_N a été introduit à la question 2) et que m est inférieur à M_N .

Définition : Afin de numéroter l'ensemble des éléments d'une liste gauche $\ell = ((a_n)_{0 \leq n < N}, e)$, nous parcourons d'abord les éléments de l'arbre exceptionnel e dans l'ordre préfixe, puis nous parcourons les éléments des arbres $a_0, a_1, \dots, a_{N-1}$ dans l'ordre préfixe. Les numéros sont attribués aux valeurs rencontrées par ordre de première rencontre.

À titre d'exemple, la figure 4 donne les représentations sous forme de listes gauches de deux listes d'entiers :



FIGURE 4 – Listes gauches correspondant respectivement aux listes d'entiers $[0; 3; 42; 8; 12]$ et $[23; 0; 3; 42; 8; 12]$

Nous adoptons le type suivant :

```
1 struct ListeGauche {
2     int hauteur_e;
3     arb* extra;
4     int nb_arbres;
5     arb** arbres;
6 };
typedef struct ListeGauche lg;
```

Le champ `hauteur_e` contient la hauteur p de l'arbre exceptionnel e . Le champ `extra` est un pointeur vers la racine de l'arbre exceptionnel e . Le champ `nb_arbres` contient l'entier N . Enfin, le champ `arbres` désigne un tableau de N pointeurs vers les arbres $(a_n)_{0 \leq n < N}$ qui a été alloué dynamiquement.

Attention ! Les structures de type `lg` seront dans la suite le plus souvent passées par valeur, pas par adresse. Prêtez particulièrement attention au prototype de chaque fonction.

1. Les listes binaires gauches, ou *skew binary lists*, ont été inventées par Eugene Myers en 1983.

3.1 Opérations simples sur les listes gauches

20. Écrire en C une fonction `lg lg_init(int nb_arbres)` qui renvoie une liste gauche de cardinal nul sur `nb_arbres` arbres.
21. Écrire en C une fonction `int lg_card(lg l)` qui calcule le cardinal m de la liste gauche ℓ .
22. Écrire une fonction `arb* lg_trouve(lg l, int k)` qui renvoie le k^{e} nœud de la liste gauche ℓ . On supposera que k est un indice valide ($0 \leq k < m$).
On pourra se contenter de pseudo-code.
23. Calculer la complexité en temps dans le pire des cas de `lg_trouve(l, k)` en fonction de la capacité maximale M_N de la liste gauche ℓ .
24. Écrire en C une fonction `int lg_acces(lg l, int k)` qui renvoie la k^{e} valeur de la liste gauche ℓ ainsi qu'une fonction `void lg_modif(lg l, int k, int v)` qui remplace la k^{e} valeur de la liste gauche ℓ par la valeur v .

3.2 Ajout et suppression en tête de liste gauche

25. Soient v une valeur entière et $\ell = ((a_n)_{0 \leq n < N}, e)$ une liste gauche de cardinal m , avec $m < M_N$, qui contient les éléments $v_1, \dots, v_m$ dans cet ordre. Décrire, en fonction de la liste gauche ℓ , la liste gauche $\ell' = ((a'_n)_{0 \leq n < N}, e')$ de cardinal $m + 1$ dont les éléments sont $v, v_1, \dots, v_m$ dans cet ordre.
26. Les fonctions `lg_access` et `lg_modif` accèdent à la liste gauche en argument par valeur. Justifier (on pourra s'aider d'un schéma de la mémoire) qu'il n'est pas possible de faire de même avec une fonction `lg_empile` qui insère une valeur en tête de liste.
27. En déduire le principe d'une fonction `bool lg_empile(int v, lg* l)` réalisant l'insertion de la valeur v en tête de la liste gauche ℓ pointée par `l`, où le booléen résultat vaut `true` si l'ajout a eu lieu et `false` si un débordement de capacité de la liste se produit.
On se contentera de pseudo-code, qu'on n'hésitera pas à simplifier en faisant référence à des opérations ressemblant aux questions précédentes.
28. Déterminer la complexité en temps dans le pire des cas de la fonction `lg_empile`.
29. Donner le principe d'une fonction `bool lg_depille(int* w, lg* l)` réalisant le retrait de l'élément de tête de la liste gauche ℓ pointée par `l`, et affectant cet élément à l'adresse w . Le booléen résultat vaut `true` si le retrait a eu lieu et vaut `false` si l'opération a échoué en raison d'une liste vide. On ne demande pas le code complet.
Comme dans la fonction 26, on se contentera de pseudo-code simplifié.
30. Donner la complexité en temps dans le pire des cas de la fonction `lg_depille`.
31. Discuter la possibilité d'obtenir une fonction plus rapide dans certains cas à la question 30, quitte à ajouter un champ dans la définition du type `lg`. On pourra s'inspirer de la question 8. Le pire cas est-il affecté?
32. Soit N un entier et $\ell = ((a_n)_{0 \leq n < N}, e)$ une liste gauche pointée par `l_addr`. Discuter la possibilité de modifier en place et avec une faible complexité en temps la liste gauche ℓ de sorte que le nombre d'arbres N devienne $N + 1$ et que les mêmes éléments demeurent dans la liste.